

University of Southern Denmark

Campus Odense

Department of Mathematics and Computer Science

Implementation of the Implicit Association Test

Bachelor's Project in Computer Science
BADM500

Authors:

Alex Vincent Hirsch Jeppsson — `aljep23@student.sdu.dk`

Martin Kuntz Jacobsen — `marja23@student.sdu.dk`

Supervisor:

Peter Mayer



June 2026

1 Abstract

This thesis investigates how a web-based Implicit Association Test (IAT) can be implemented so that response-time measurements are minimally affected by the application itself. The project treats the IAT as a timing-sensitive software problem rather than as a validation of the psychological test. The system generates a deterministic session plan before the test begins, loads the required stimuli on the client side, and then runs the timed trials locally in the browser. The implemented system consists of a TypeScript single-page front-end, a FastAPI back-end, a SQLite database, YAML-based IAT definitions, a Selenium Grid evaluation tool, and an offline analysis pipeline.

The evaluation uses automated sessions that run through the participant-facing application in the same way an ordinary user would. The benchmark compares different stimulus types, response delays, execution platforms, and environmental conditions. The analysis dataset contains 260 completed sessions and 22,880 trials. Every client-recorded response latency fell within the evaluator’s dispatch window. In the three baseline conditions, the standard deviation of the difference between the client-recorded latency and the evaluator’s before-input timestamp was between 0.17 ms and 0.18 ms. CPU throttling increased timing variance and dispatch overhead, while network degradation mainly increased the session duration.

The main conclusion is that the implemented client-side architecture limits avoidable measurement variance in a web-based IAT application. It does so by moving stimulus loading and network communication out of the timing-sensitive trial loop. Remaining variance is still affected by the browser, hardware, CPU load, display behavior, and input hardware. The system therefore provides a technically viable IAT application in the browser, while psychological validation remains outside the scope of this thesis.

Contents

1	Abstract	1
2	Introduction	4
3	Background	5
3.1	Implicit association	5
3.2	Client-side architecture	7
4	Related work	8
4.1	Performing psychological tests via a web application	8
5	Methodology	9
5.1	Research Strategy and Study Object	10
5.2	Experimental Design and Variable Control	10
5.3	Stimulus Management	12
5.4	Benchmark procedure	12
5.5	Environmental Simulation	13
5.6	Analysis procedure	14
5.7	Methodological Limitations	14
6	Architecture	14
6.1	Architectural Drivers	15
6.2	System Overview	15
6.3	Session Life-cycle and Data Flow	17
6.4	Deterministic Run Plans	17
6.5	Client-side State Machine	18
6.6	Timing Strategy	19
6.7	System Components and Responsibilities	19
6.8	Evaluation Architecture	22
6.9	Implementation Mapping	24
6.10	Limitations and trade-offs	25
7	Implementation	25
7.1	Back-End Session Preparation	25
7.2	Front-End Flow	26
7.3	Timing and Block Uploads	27

7.4	Persistence and Scoring	27
7.5	Evaluation Tooling	28
7.6	Build and Dependency Management	28
8	Evaluation	29
8.1	Experimental Setup	29
8.2	Condition-Level Results	29
8.3	Latency and Timing Variance Benchmarks	31
9	Discussion	33
9.1	Interpretation of Results	33
9.2	Limitations and Future Work	34
10	Conclusion	35
	Bibliography	36
11	Appendices	37
11.1	Source code and evaluation data	37
11.2	Usage of GenAI	38

2 Introduction

Modern web applications can execute a substantial part of their logic inside the browser. For applications that offer time-sensitive services, this is important because it allows for high-precision computations (Leeuw and Motz 2016; Bridges et al. 2020). Having the local client perform computations rather than the server reduces dependence on network throughput during the timing-critical parts of the application.

Since network connections between two devices are prone to fluctuations, offloading the computations to the client reduces the effect of those fluctuations on the timing-critical processes, as the need for retransmitting data is eliminated (Rotem-Gal-Oz 2006). This enables the web application to offer time-sensitive services.

One area that benefits from time-sensitive services is psychology, because the ability to perform accurate participant measurements via a network-based application means that researchers are able to devise tests that can be run on the participant’s own devices, making them more accessible (Rodd 2024). However, this also introduces technical challenges, since the recorded timing measurements may be affected by browser behavior, hardware differences, input devices, and network conditions (Rotem-Gal-Oz 2006; Anwyl-Irvine et al. 2021).

This is especially relevant for tests that rely on rapid responses, where small timing differences can affect the interpretation of the results (Greenwald et al. 1998; Greenwald et al. 2003). A subset of psychological tests that measure instinctive aspects of participant behavior relies on accurate accounts of how the participant acted instinctively throughout the test. Because some human instincts express themselves almost instantaneously, the measurement method must account for this. One of the behavioral tests whose results depend on accurate readings of the participant’s rapid responses is the Implicit Association Test, often abbreviated as the IAT.

This thesis explores how a deterministic client-side architecture can limit avoidable measurement variance by evaluating the timing behavior of a web-based IAT implementation. The explicit research question is:

How can a deterministic client-side architecture minimize measurement variance in web-based Implicit Association Tests across heterogeneous hardware and varying network conditions?

The thesis is structured as follows. The background chapter introduces the IAT, reaction-time measurement, and relevant web application concepts. The related work chapter discusses existing knowledge on IAT scoring, web-based psychological testing, and timing accu-

racy in browsers. The methodology chapter describes the research approach and evaluation strategy. The architecture chapter presents the system design and explains how deterministic planning, preloading, client-side execution, and persistent session data support the research question. The implementation chapter describes how the architecture was realized in the application. The evaluation chapter presents the benchmark results, while the discussion chapter interprets the findings, limitations, and possible future work. Finally, the conclusion summarizes the answer to the research question.

3 Background

3.1 Implicit association

In psychology, implicit associations refer to an individual’s unconsciously assigned associations with certain objects or concepts. These biases are shaped by an individual’s cultural upbringing or other exposure to culture (Greenwald and Banaji 1995).

This pattern has been theorized to be an expression of an evolutionary trait, since being able to judge a target almost instantly, without direct experience with the subject, allows the subconscious to favor safety during moments of uncertainty (Haselton et al. 2015). This trait is evolutionarily beneficial since it allows for snap-decision-making informed by prior lived or culturally ingrained experiences.

Importantly, these associations can often be incorrect or otherwise faulty, especially when paired with the mind’s predisposition towards grouping and categorizing objects and concepts (Tajfel 1970; Goldstone et al. 2003). These implicit associations have been shown to be malleable enough that they can be momentarily adjusted by having experiences that contradict these associations (Dasgupta and Greenwald 2001; Forscher et al. 2019). It should be noted that the unconscious association is strong enough that it often rejects and overrules associations that the conscious mind accepts, meaning that an individual may be unaware of the biases they exhibit (Greenwald et al. 1998).

One of the ways in which these consequences present themselves in the modern world is in the form of unfounded stereotypes targeting social groups (e.g., racial, ethnic, sexual, gender, or age categories), which can cause an individual belonging to any of those categories to be unfairly discriminated against or otherwise harmed (Cuddy et al. 2007).

Greenwald et al. (1998) discuss the possibility of ascertaining an individual’s implicit bias against a target concept through a test. The Implicit Association Test measures whether a subject has a tendency to associate certain concepts with specific social groups.

3.1.1 Design of the IAT

In its original design, the IAT is performed by having the participant complete a set of blocks, each containing a fixed number of trials (Greenwald et al. 1998). Each block begins with the subject being instructed on which associations they should link to either group, with the instructed pairings changing at the start of each block. Some of these configurations are deemed congruent, while the remainder are incongruent. A trial consists of the subject being presented with a stimulus, either visual or textual, which they have to link to the instructed category. Visual stimuli are pictures depicting members of such a group, while textual stimuli refer to words with a clear positive or negative association attached.

The completed test yields a score that shows whether a subject has a positive or negative bias towards a tested group. This score is named the *D*-score (Greenwald et al. 2003).

Some of the original authors later revised the method for calculating the *D*-score (Greenwald et al. 2003). They recommended using the D2 variant formula for calculating the *D*-score.

This variant differs from the original in that it includes the trials from blocks 3, 4, 6, and 7. It discards trials whose response time exceeded 10,000 ms or was under 400 ms. If more than 10% of trials' response times were under 300 ms, the entire session would be rejected (Greenwald et al. 2003).

The difference between the incongruent and congruent mean response times was then divided by the pooled standard deviation. The formula used for calculating the *D*-score is therefore:

$$D = \frac{\text{Mean}_{\text{incongruent}} - \text{Mean}_{\text{congruent}}}{\text{Pooled Standard Deviation}}$$

3.1.2 The IAT as a web application

The measurement of bias, the *D*-score, is calculated from how quickly the subject is able to pair the stimuli with the instructed category. This means that for the test to provide the most accurate readings, it must exhibit a high degree of responsiveness in measuring the user's inputs. When implementing the IAT digitally, it is therefore paramount that the readings remain unaffected by noise caused by the client's miscellaneous background processes or varying network conditions (Anwyl-Irvine et al. 2021).

To obtain accurate measurements from a trial during the IAT, there are two factors that have to be controlled. Firstly, the participant's inputs must be read and logged as soon as possible after a key press. Secondly, the trial must not proceed before the stimulus is correctly displayed to the participant. Both of these factors are simple to control when the

IAT is implemented in an offline environment, but when the IAT is implemented as a readily available online test, they become more difficult to control.

Continual network transfers between a client and a server are prone to disruptions that can add wait time to the transfers, which is also called latency. This makes methods that run over a network connection unreliable for performing the high-precision measurements needed during the IAT. Therefore, the implementation should ensure that these measurements are performed in a more controlled environment, which can be achieved by allowing the application to use the client’s local hardware.

3.2 Client-side architecture

Client-side architecture aims to limit reliance on a continuous network connection between the client and the server by running core logic directly on the client device. This allows for the development of web applications that perform time-sensitive services, as they circumvent transmission latency caused by network fluctuations (Rotem-Gal-Oz 2006).

To achieve this architecture, the front-end presentation layer must not be directly reliant on the back-end to perform its core functions.

While relying on the individual client’s hardware for these timing-critical measurements imposes new challenges of its own, it also allows for the deployment of methods, such as high-accuracy timestamps enabled by local hardware (MDN Web Docs Contributors 2026c).

3.2.1 JavaScript

JavaScript allows developers to use the client’s browser to access local hardware.

JavaScript utilizes an event loop, which is a queue of incoming commands or tasks that are executed sequentially (MDN Web Docs Contributors 2026b). These tasks may include processing keyboard input events or updating the webpage’s displayed contents. The single-threaded nature of JavaScript means that tasks further back in the queue won’t be processed until the preceding ones have been (Ecma International 2025, Clause 9.7–9.8). Since the webpage also relies on the event loop to continually render its contents, a particularly long or intensive task might delay the rendering of a new frame (MDN Web Docs Contributors 2026a).

3.2.2 Hardware considerations

Having the web application rely on the client’s own hardware introduces some uncertainty regarding the validity of the test’s results. Since the test has to be readily available via the internet, it is accessible on a large variety of devices with different hardware configurations or other computational limitations, which could potentially skew the measurements of the response times (Bridges et al. 2020).

3.2.3 Network considerations

To benefit from a client-side architecture, the implementation must consider how data is sent from the server to the client, and vice versa. If, during the IAT, the results of each trial were transmitted back to the server as they were completed, it could induce latency (Rotem-Gal-Oz 2006). Continual transmission would fill the web application’s event loop with network requests, potentially causing the other tasks to be delayed (MDN Web Docs Contributors 2026b). The same could occur if the stimulus for each trial were retrieved from the server at the start of the trial. Developing an appropriate structure for data transmission is, therefore, an important part of implementing a client-side architecture.

Standard architectural patterns for web-based psychological tests recommend that the entire set of stimuli be retrieved and stored locally before the test begins. By storing trial results locally and transmitting them back to the server as a single bundled payload at the end of the test, the implementation minimizes network-induced artifacts while performing timing-critical measurements (De Leeuw 2015).

4 Related work

4.1 Performing psychological tests via a web application

The act of facilitating comparable types of participant responses off-site via a web application, such as those generated in lab environments overseen by researchers, has been studied by Rodd (2024). The paper details the issues that stem from running behavioral tests off-site and provides guardrails to minimize their impact.

According to the study, the participant’s performance is directly tied to how well they understand the test and their role in it. There should be a concerted effort to properly explain to the participant how they should interact with the test before, during, and after the test.

De Leeuw (2015) introduces jsPsych, which is a JavaScript library designed for implementing behavioral experiments inside a browser environment. This library is especially well-suited for tests that measure a participant’s responses to stimuli, including the Implicit Association Test.

However, the scope of this library does not extend beyond the immediate JavaScript implementation, meaning that it does not provide the entire architecture for a web application but merely the client-side execution logic (De Leeuw 2015). When using the jsPsych library, developers must design their own back-end infrastructure to support this front-end.

For online behavioral tests, it has been shown that using JavaScript to implement high-precision timestamping is capable of delivering response-time measurements whose accuracy is comparable to that of on-site lab environments, provided that the code accounts for underlying latency-inducing factors (Leeuw and Motz 2016; Bridges et al. 2020).

One specific method of measuring response time is to combine `performance.now()` to create a high-precision timestamp with a `keydown` event listener to stop the measurement when the input has been observed, as it has been shown to provide consistently accurate measurements (Reimers and Stewart 2014; MDN Web Docs Contributors 2026c).

For tests that hinge on a subject’s response times to stimuli, the implementation should aim to start the timestamp at the same time the webpage’s elements are redrawn. Using the `requestAnimationFrame()` command (rAF) allows for making function calls to be synchronized with the webpage’s next redraw, which is based on the monitor’s refresh rate, meaning that the timestamp is started immediately before the webpage is redrawn. While this means that the timestamp isn’t started exactly as the stimulus is presented, it is still an easily implemented method that can achieve accurate results (Gao et al. 2020).

Lukács and Gartus (2022) explore the different methods for synchronizing web-page redraws via the rAF command. They found multiple methods, such as single-rAF, double-rAF, and rAF-loop. Double-rAF issues the display command in the first rAF callback and then uses the timestamp from a second nested rAF callback for timing. The study found little to no improvement of double-rAF over single-rAF, whereas rAF-loop produced the best overall precision and was recommended by the authors.

5 Methodology

This thesis uses a design-and-evaluation approach. The main research contribution is a working software artifact and the evaluation of its timing behavior. The artifact is a web-based

Implicit Association Test platform designed to reduce avoidable variance during the trials. The IAT provides a timing-sensitive use case where the project focuses on the software architecture needed to make such a test run reliably in a browser rather than on the psychological validity of the test.

5.1 Research Strategy and Study Object

This focus shapes the research strategy. Since the project investigates the behavior of a software artifact, the evaluation is based on controlled browser sessions. The goal is not to measure implicit bias, but to examine whether the implemented architecture reduces the timing variance introduced by the application itself.

The study, therefore, focuses on the runtime behavior of the system. The project studies how the application behaves when IAT sessions are executed in the browser under different benchmark conditions.

Because of the technical focus, the evaluation uses automated browser sessions with synthetic responses. This allows sessions to be repeated across different benchmark conditions while keeping the responses controlled. No human participant studies were performed with the system because that is not necessary for evaluating the software itself.

5.2 Experimental Design and Variable Control

The evaluation is designed to measure the behavior of the participant-facing application itself. Every benchmark session therefore uses the same front-end, back-end, SQLite database, and application flow. The evaluation harness does not bypass the participant application. Instead, it interacts with the rendered page through key presses and clicks. The evaluation uses a read-only interface to get information about the application state. This interface is part of the regular application.

The independent variables are grouped into four categories.

- Stimulus variation, varied across text/text, image/text, and image/image IATs.
- Response delay, varied between 410 ms and 1000 ms.
- Environmental pressure, varied through CPU throttling and network degradation.
- Execution platform, varied across Linux, Windows, and Android environments, as well as Chrome and Firefox browsers.

The dependent variables are divided into timing metrics and validation metrics. The primary timing metric is the latency, which is the recorded response time for each completed trial. This is the primary metric because the research question concerns whether timing variance can be reduced.

The validation metrics are used to verify whether the application behaves correctly during the benchmark. Session duration is used to observe the end-to-end duration of the complete IAT session. Platform consistency is used to compare whether timing results remain similar across tested browsers, operating systems, and devices.

Several factors were kept fixed within comparable benchmark runs. The same plan seed was used so that sessions had the same trial order. The correct response was always selected, and the response delay was measured relative to the moment when the front-end reported that the trial had started. This keeps the responses consistent across runs and makes benchmarks easier to compare.

Table 1 summarizes the benchmark conditions used in the evaluation. The table includes the baseline conditions, the longer response delay, CPU throttling, and network degradation. For each condition, the benchmark was run ten times per worker. This produced repeated observations while keeping the evaluation practical to run.

Condition	IAT	Delay	Primary purpose
Text/text baseline	Female/Male Science/Liberal Arts	410 ms	Baseline timing where both category pairs use text stimuli.
Image/text baseline	White/Black Pleasant/Unpleasant	410 ms	Baseline timing where one category pair uses images and the other uses text.
Image/image baseline	Young/Old Female/Male	410 ms	Baseline timing where both category pairs use image stimuli.
1000 ms delay	White/Black Pleasant/Unpleasant	1000 ms	Image/text benchmark with longer response delay.
CPU throttling	White/Black Pleasant/Unpleasant	410 ms	Image/text benchmark with browser CPU throttling at a rate of 10.
Network degradation	White/Black Pleasant/Unpleasant	410 ms	Image/text benchmark with 500 kbps download, 220 ms latency, and 500 kbps upload.

Table 1: Benchmark conditions used in the evaluation.

5.3 Stimulus Management

The visual stimuli used in the evaluation were generated using AI tools. This was done because the project evaluates the timing behavior of the application and not the psychological validity. The stimuli are therefore only needed to test the difference between image and text stimuli.

Using generated stimuli removes licensing and copyright constraints, allowing the application to be publicly distributed without concern for content restrictions. This improves reproducibility because external researchers can deploy the exact same software environment without any legal barriers.

5.4 Benchmark procedure

The benchmark procedure is designed to follow the normal application flow as closely as possible. Each benchmark uses the ordinary application, but does so in evaluation mode. Evaluation mode uses the same application flow as an ordinary session, except that it allows the benchmark to use a fixed plan seed.

Each benchmark session follows the same general flow:

1. The browser is started on each worker with the selected benchmark configuration.
2. The browser enters the specific IAT using a fixed plan seed.
3. The automation client starts blocks when the application is ready to advance.
4. For each trial, the automation client waits until the trial is active and then sends the correct response after the configured delay.
5. The benchmark is repeated ten times for each worker.
6. Benchmark results such as the evaluator response window, browser, platform, and session key are stored for later analysis.

The configured response delay should be understood as a best-effort delay. The runner waits for the front-end to report that a trial is active and then schedules the response using JavaScript timing. This keeps response behavior repeatable across runs, but the exact dispatch time is affected by browser scheduling, CPU load, and the platform.

The automation client does not directly alter the internal state of the application. It only observes the front-end state and interacts with the rendered page through normal input

events. This is important because the evaluation should measure whether the implemented flow is correct, rather than simulating it.

The response delay is tied to the application’s state instead of being based on a fixed delay, a page load, or a visual change. This makes the automated response closer to the timing model used by the application itself.

5.5 Environmental Simulation

The evaluation includes conditions where the browser is run under artificial CPU and network pressure. These conditions are applied at the browser level, which means that the benchmark still executes the same application code and flow. This is only done for the evaluated Chromium-based browsers, because they support the emulation interface directly.

5.5.1 CPU Throttling

CPU pressure is simulated using CPU throttling with a slowdown rate of 10. This is relevant because rendering, event handling, state transitions, and JavaScript timing are influenced by CPU pressure. It is therefore expected that this condition can result in higher variance.

5.5.2 Simulated Network Degradation

Network degradation is simulated with reduced throughput and added latency. The network profile uses 500 kbps download, 500 kbps upload, and 220 ms latency, with caching disabled.

Network degradation is applied before the application is loaded. This makes the condition closer to what a participant would experience when running on a slower connection. The purpose is to show that the application handles image loading before trials and does not fetch stimuli during the timed trial loop.

5.5.3 Longer Delay Stability Evaluation

The increased delay of 1000 ms acts as a longer stability check. Increasing the delay from 410 ms to 1000 ms increases the session length without changing the session plan. This allows the evaluation to observe whether timing variance becomes worse as the browser remains active for longer periods.

5.6 Analysis procedure

The evaluation data is collected from two sources. The evaluation runner stores information such as condition, platform, browser, executed response delay, and the session key produced during the run. The back-end stores session metadata, the run plan, and the recorded responses in the SQLite database. During analysis these two sources are joined using the session key so that the benchmark can be evaluated against the persisted application records.

The main timing analysis is based on the response window recorded by the evaluator. For each response, the evaluator records the timestamp before and after sending the input event. This gives the interval in which the automated response was executed, rather than treating the configured delay as an exact execution time. The interval is then compared against the time recorded by the participant-facing client. This is important because the configured response delay is a best-effort delay.

The analysis also includes secondary measurements. Session duration is used to inspect the end-to-end duration of the completed IAT sessions. Platform consistency is used to compare whether timing results remain similar across browsers, operating systems, and devices.

The analysis uses descriptive statistics to compare benchmarks. For each condition, the analysis reports the number of observations, mean, standard deviation, minimum, and maximum.

5.7 Methodological Limitations

The methodology has two main limitations. First, the evaluation uses automated responses rather than human participants. This is useful for evaluating the timing variance introduced by the software, but does not evaluate the psychological effects. Second, the results are limited to the tested browsers, operating systems, devices, and environmental conditions.

6 Architecture

The purpose of the application is to run IATs in web browsers while reducing the measurement variance introduced by the software itself. Since the IAT relies on reaction times, the architecture must avoid making network requests, server-side decisions, or content loading during the timing-critical part of the test. The central goal of the architecture is therefore to prepare a deterministic run plan before the timed phases and then make the browser execute the plan without interference.

The system is divided into layers not only to improve maintainability, but also to separate responsibilities. The browser is responsible for the timing-sensitive loop of the test. The application server prepares sessions, persists session data, and calculates scores. The content catalog defines IATs, while the persistent store records generated run plans and observed participant events. This makes it possible to reproduce scoring and analyze completed sessions later.

6.1 Architectural Drivers

The design is shaped by five main architectural drivers.

Measurement validity. The timed phase of the test should measure the participant's response, not the latency of network requests or the time to fetch stimuli. The client must therefore have the relevant trial data before executing the trial. The most important factor here is measurement variance, not necessarily latency, since latency can be accounted for.

Determinism. Each IAT session must be reproducible so that the test itself does not become an uncontrolled variable while evaluating the system. The entire plan must be reproducible so that the test can be compared under different conditions.

Flexible test definitions. The application must support different IATs, allowing changes in category pairs and stimuli, without changing the runtime code. This is needed because the system should be able to measure different categories of implicit associations.

Device diversity. The same test flow must work on desktop and mobile clients. Different devices may use different input methods, such as keyboard input on desktop devices and touch input on mobile devices. Both input methods should be handled through the same internal logic, so that the session flow remains comparable across devices.

Reproducible evaluation. The architecture must support controlled benchmarking across different hardware and network conditions. The evaluation system should use the same runtime path as ordinary participant sessions. Otherwise it would not measure the behavior of the implemented web application.

6.2 System Overview

The system is organized into six architectural parts.

- the browser client, which renders the participant interface and executes trials
- the application server, which exposes the public API, creates session plans, stores data, and calculates scores
- the content catalog, which contains IAT definitions and stimulus resources
- the persistent session store, which records plans and participant responses
- the evaluation runner, which automates real browser sessions under controlled conditions
- the offline analysis pipeline, which joins evaluation output with persisted session data

Figure 1 shows the main components and their relationships.

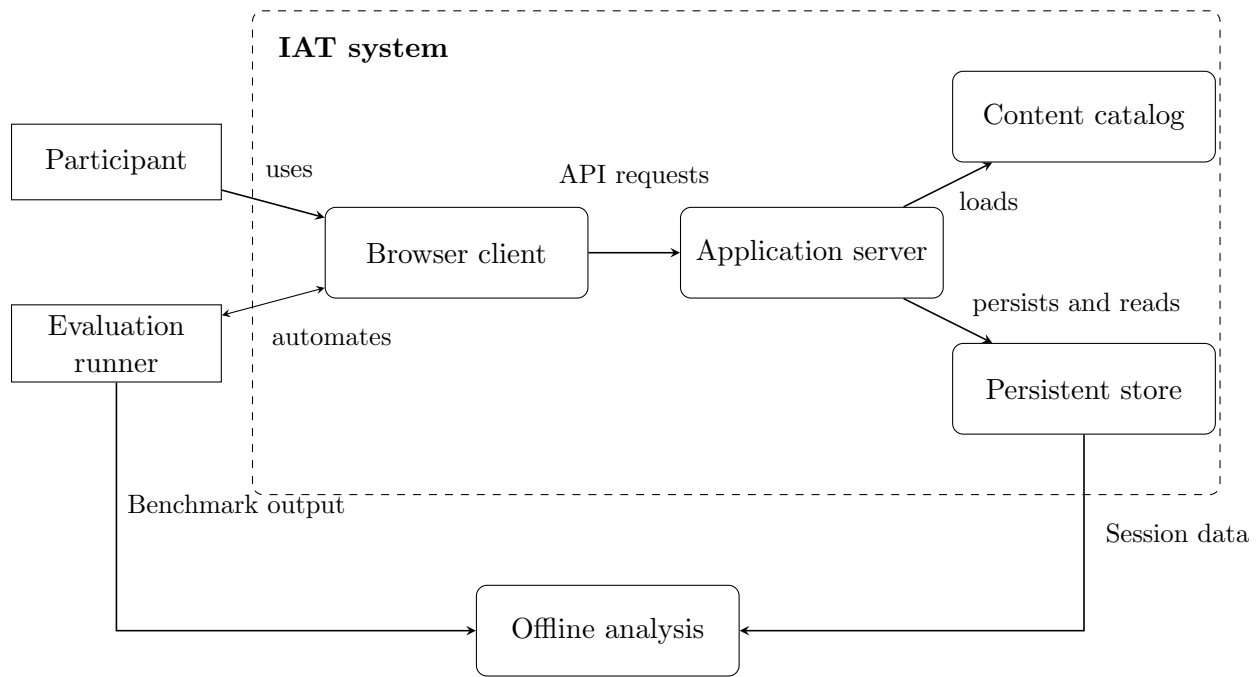


Figure 1: C4 level 2 container view of the IAT system.

The browser client and application server form the participant-facing runtime. The evaluation and analysis components are outside the normal participant path, but they are still architecturally important because they measure whether the application reduces timing variance.

6.3 Session Life-cycle and Data Flow

Participant sessions are structured so that the network-dependent work is placed outside the timing-critical code. Before the test begins, the server creates and stores a deterministic run plan, which the client loads while preloading all stimuli required for the test. During each block, the trials are executed locally in the browser. After each block is completed, the responses are uploaded to the server.

The data flow is therefore divided into three phases:

1. Preparation: the server creates a deterministic plan and the client preloads the required content.
2. Measurement: the client executes each block of timed trials locally and records response events without requiring server communication during individual trials.
3. Block persistence and scoring: after each completed block, the client uploads the measured block data to the server. Once all blocks are uploaded, a score is requested from the server.

This separation is the main mechanism for reducing network-induced measurement variance. Network requests are required for setup, persistence, and scoring. However, during the active trial loop the application is not dependent on fetching or loading stimuli or plans. The application is therefore not fully offline during the test, but the timing-critical trials are offline.

6.4 Deterministic Run Plans

The application supports different IATs through test definitions. A test definition describes the test title, description, two category pairs, the labels in each category, and the stimuli belonging to those categories. The run plan is generated from this definition.

A run plan defines the full session. This includes the order of blocks, labels assigned to each response side, stimuli, and the correct response. Once this plan has been created, the client can execute the trials without the need to request further information from the server.

The run plans are generated deterministically from seeds, which means that the same input definition and seed produce the exact same block and trial structure. This is important because the test structure should not become an uncontrolled source of variance when sessions are compared across hardware, browsers, and network conditions.

6.5 Client-side State Machine

The browser client is created around a state machine. Figure 2 shows the stages and their transitions.

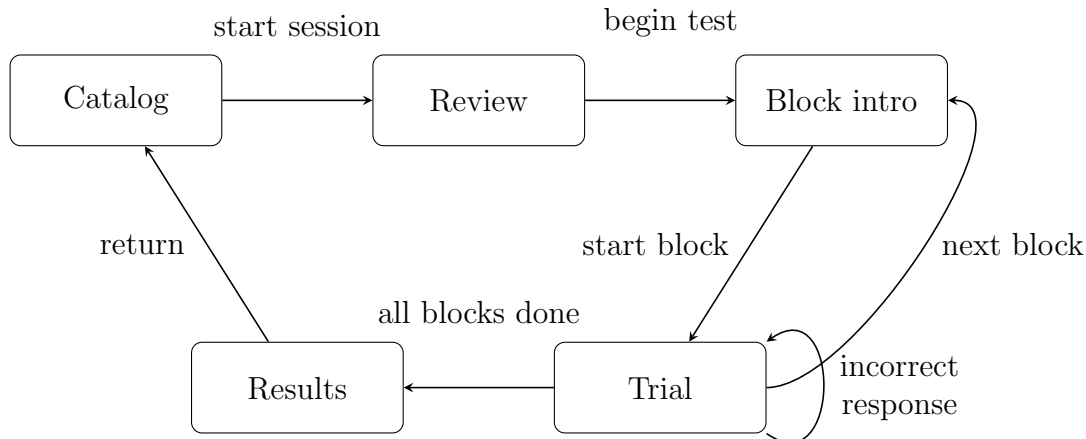


Figure 2: Front-end session state machine.

The state machine has five stages. In the catalog state there is no active session and the participant can choose a test. In the review stage the selected test and its stimuli are shown, and the participant is not allowed to proceed until the stimuli are loaded. The block-introduction state presents the response mapping for the next block along with the layout so that there is no layout change once the trial starts. The trial state presents the stimulus and records the participant responses. This block/trial loop runs until all blocks have been completed. The result state then uploads the remaining block and requests the score before showing it to the participant.

This state machine is important because it localizes the timing-sensitive behavior. The trial state records response times while the other states handle preparation, transitions, uploads, and presentation.

The client supports both keyboard input and touch controls. Regardless of input type, the response is stored in the same way, with the selected side and response time. This allows desktop and mobile sessions to follow the same test flow, while differences between keyboard and touch input can still be considered during evaluation.

6.6 Timing Strategy

The client records timing locally in the browser. This is possible because the run plan has already been created and loaded by the client along with any required stimuli. Individual trials therefore do not depend on network requests or server decisions. For each trial, the response time is measured relative to the moment the stimulus is presented.

When the participant responds, the client stores the elapsed time and the selected side. Incorrect responses are also stored, but only valid responses advance the trial. This preserves all responses and allows for richer data analysis.

This timing strategy avoids unnecessary variance due to network access and late content loading. It does not, however, remove variance from hardware, display refresh rate, or input devices.

6.7 System Components and Responsibilities

This subsection describes the main runtime components and how they support the architectural goals.

6.7.1 Browser Client

The browser client presents the IATs to the participants and executes the timed trials. It manages input handling, session state, rendering, stimulus loading, and communication with the server through the API.

Its most important responsibility is to keep the active trial loop local. Once the session plan and stimuli have been loaded, the client can run trials, record responses, and advance within blocks without server communication. This supports the timing strategy by keeping network requests outside of the trials.

Figure 3 shows the main client-side components. The session flow updates the runtime state, the rendering layer presents the current state to the participant, the input controller handles participant input, and the image cache preloads stimuli that can then be used during the test.

6.7.2 Application Server

The application server handles the responsibilities that should not belong to the browser loop. It serves the browser client, exposes the catalog and stimuli, creates deterministic plans, receives block completions, and calculates scores from persisted data.

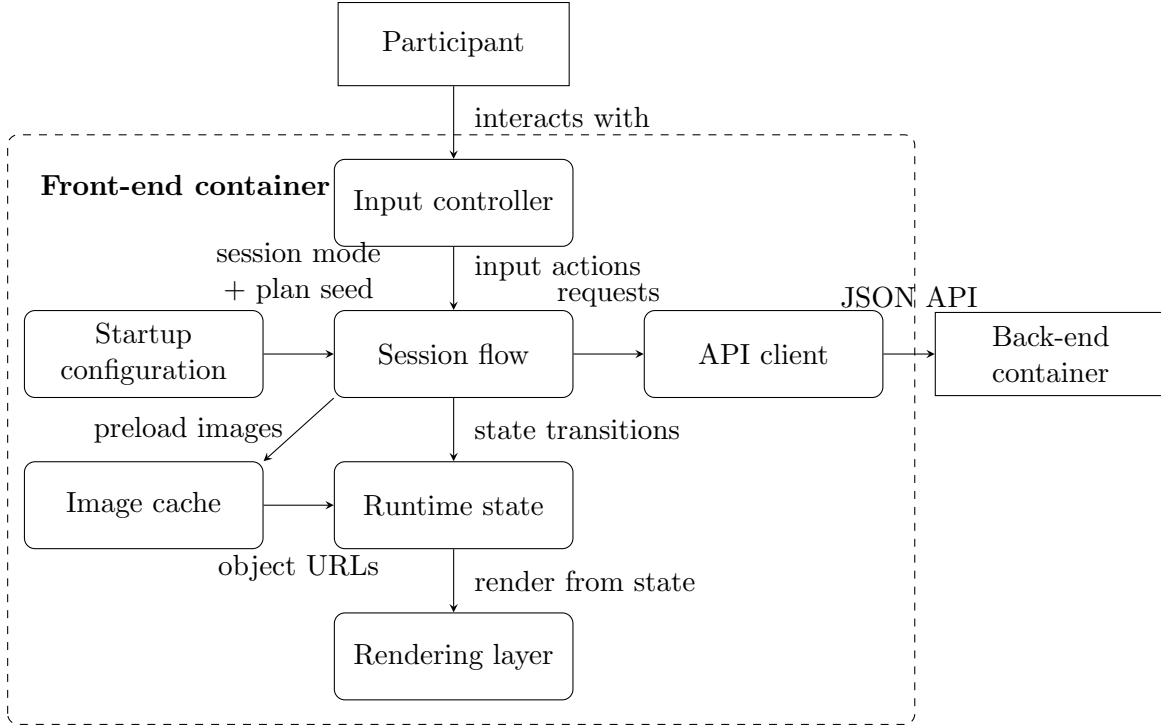


Figure 3: C4 level 3 component view of the front-end container. Startup configuration resolves URL-derived session inputs, session flow drives the runtime state, image preloading stores object URLs in state, and the rendering layer reads that state while the API client handles back-end requests.

The server is kept outside of the active trial loop. Its role is to prepare the session before the timed trials and to persist data after it has been measured. The data uploads are done after each completed block so as not to interfere with the timing-critical code.

Figure 4 shows how the application server is divided into routing, services, and repositories. These components separate the responsibilities of serving assets, catalogs, stimuli, sessions, and scores.

6.7.3 Content Catalog

The content layer is an auxiliary layer that is responsible for storing all of the material required to instantiate an IAT. Rather than embedding the test-specific content directly into the application logic, it is externalized into structured resources. The content consists of IAT definition files and the stimuli referenced by those definitions.

The IAT definition files describe the structure of each test, including description, categories, and stimuli. The stimuli are resolved from these definition files, and both image and text stimuli are supported. The IAT definition files are not exact session plans, as those are

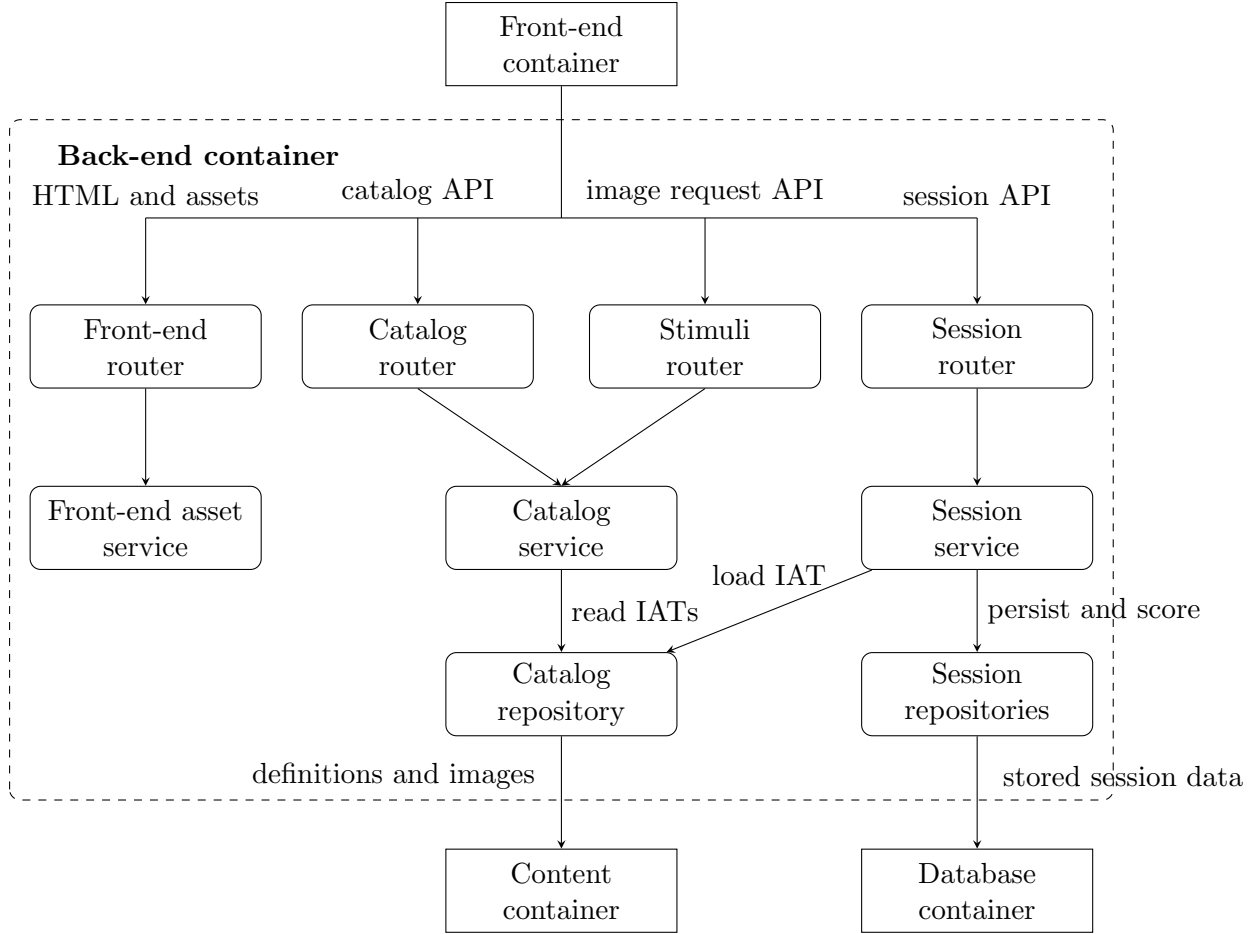


Figure 4: C4 level 3 component view of the back-end container. The back-end serves the front-end, publishes test content, creates deterministic sessions, and persists completed sessions for scoring.

generated deterministically by the back-end.

The content layer is intentionally declarative and read-only during application execution. It does not contain business logic or runtime behavior, it only provides the configuration and content for the back-end to create IATs. This separates the concern of defining implicit association tests from the rest of the system, allowing the application logic to remain constant while new tests can be introduced, revised, or removed.

This design improves maintainability and reproducibility, since IAT definitions are stored as files that can be versioned and traced over time. Figure 5 shows how the back-end interacts with the content layer’s components.

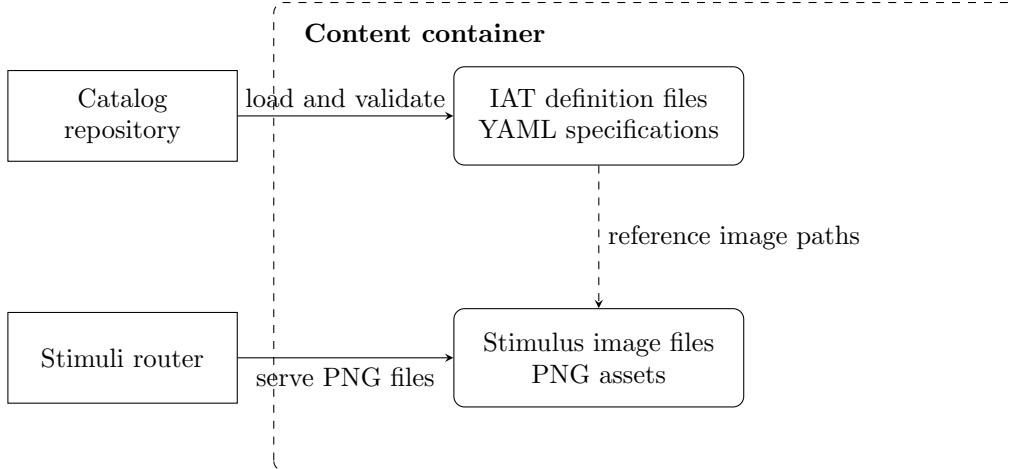


Figure 5: C4 level 3 component view of the content container. The content container holds the IAT definitions and the stimuli used by those definitions.

6.7.4 Persistent Store

The persistent store records both the planned runs for sessions and the observed participant responses. Rather than only storing the final score or preprocessed results, the database keeps the session metadata, block plans, block labels, trial plans, and trial events. This allows a completed session to be reconstructed and scored from the stored data, and allows alterations to the scoring function without invalidating the data.

This design ensures that scoring and analysis are based on what was actually assigned and recorded during the session, instead of relying on the current version of the content catalog.

Figure 6 shows how the database layer’s components interact with each other. It also illustrates how an incoming session’s contents are separated into the database’s different tables, and shows how those tables are hierarchically structured.

6.8 Evaluation Architecture

The project contains a dedicated evaluation system that benchmarks the IAT application through browser automation rather than isolated micro-benchmarks. This allows for evaluation of the end-to-end system from front-end to back-end, rather than only confirming that individual features work in isolation while the system as a whole could still be suboptimal.

The purpose of the evaluation code is to run IAT sessions in a controlled and reproducible way across different machines, while still using the same front-end, back-end API, and persistence store that a real participant session uses. Benchmark definitions use the same resource-driven approach as the content layer. The configuration describes the IAT to run,

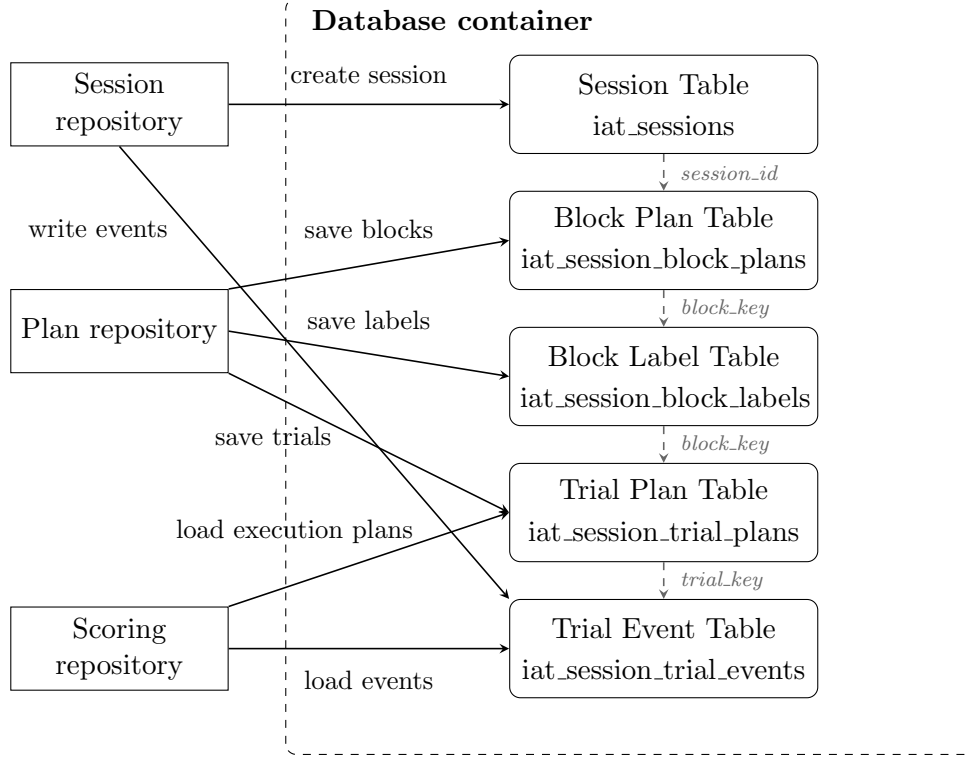


Figure 6: C4 level 3 component view of the database container schema hierarchy and data pipeline.

the seed for generating the plan deterministically, the number of sessions, and the environment constraints. This keeps the experiments reproducible and makes it possible to revise benchmarks without altering the application.

The evaluation runner is responsible for taking a benchmark configuration and turning it into browser sessions. The evaluation runner uses a distributed browser-worker setup where the runner discovers available workers, assigns the benchmark to each worker, and collects the resulting data. This allows behavior to be compared across machines, browsers, and environments. It does not replace the front-end or back-end. It only coordinates browser automation.

Each browser opens the normal participant application in evaluation mode, which differs slightly from the usual participant mode. Evaluation mode primarily serves two purposes. First, it allows the run plan to be generated from a fixed seed, such that prepared runs are deterministic. Second, it marks the session in the back-end so that it can be differentiated from ordinary participant data. Apart from this, the front-end and back-end function the same as they would for normal participants.

To ease automation, the front-end exposes a small read-only representation of its current

state. This representation gives the automation layer enough information to observe the progress of the test and determine the correct response for an active trial. Importantly, it does not expose alternate execution paths or controls. The evaluation harness cannot directly advance the state machine, respond to trials, or alter any other state using the interface. It can only observe the front-end state and then interact with the rendered page through ordinary user inputs. This ensures that the evaluated behavior is the same as in ordinary participant sessions.

The evaluation data is then collected from two sources. The runner records the benchmark context, session key, and information on the worker that performed the run. The back-end records the session data, such as the generated plan, and all trial events gathered during the session. These sources are then joined during offline analysis such that the conditions and measurements can be interpreted together.

This architecture makes the evaluation system a controlled wrapper around the application rather than a separate implementation of it. The evaluation can vary seeds, workers, and the environment while still measuring the timing behavior produced by the IAT application in ordinary sessions. This makes the results useful for evaluating the application’s timing variance.

6.9 Implementation Mapping

The architecture described here does not depend on any specific framework, but the implementation uses specific technologies. Table 2 shows which concrete technologies each architectural role uses.

Architectural role	Current implementation
Browser client	Front-end with an explicit state machine and manual rendering.
Application server	FastAPI service exposing catalog, stimulus, session, and scoring routes.
Persistent store	SQLite database accessed through SQLAlchemy, with persisted session plans and trial events.
Content catalog	YAML IAT definitions and PNG stimulus assets.
Evaluation runner	Selenium Grid-based browser automation with optional CPU or network emulation.
Offline analysis	Python analysis tool.

Table 2: Mapping from architectural roles to the current implementation.

These technologies describe the current implementation, not architectural requirements.

6.10 Limitations and trade-offs

The architecture limits avoidable measurement variance introduced by the application, such as variance from network communication and image loading. It does not, however, eliminate variance from hardware, browsers, displays, input devices, operating systems, and the difference between touch and keyboard latency. These factors are why the project includes a system for evaluating across heterogeneous environments.

Uploading after completed blocks is a trade-off between persistence and network communication. Completed blocks are uploaded between blocks, rather than at the end of the test or during trials. This avoids network interference during trials, while still allowing the system to persist progress before the full test has finished. The application is therefore not fully offline while the test is running, but it does avoid trial-time network variance.

Supporting both keyboard and touch improves device coverage, but also introduces possible differences in input latency. The system handles both input types consistently, while the difference is measured in the evaluation.

The IAT catalog is loaded at server startup, which simplifies the runtime by preventing changes to the definitions at runtime. The main trade-off is reduced flexibility, because a server restart is required before changes take effect.

7 Implementation

This application is implemented as a TypeScript single-page client, a FastAPI back-end, YAML-based IAT definitions, a SQLite database, and a Bazel build setup, as shown in Table 2.

7.1 Back-End Session Preparation

The back-end is implemented with FastAPI and served by Uvicorn. It exposes routers for serving the front-end, listing IATs, serving stimulus images, creating sessions, uploading completed blocks, and fetching scores.

The API code is split into routers, services, and repositories to keep the code maintainable and understandable. Services contain the main domain logic, such as session creation, plan generation, and scoring. Repositories contain the access logic for content files and the database. The routing layer only handles turning service responses into the correct API responses.

The IAT definitions are stored as YAML files, with each definition containing the title, description, category pairs, response labels, and the stimuli. Text stimuli are stored directly in the definition, while image stimuli refer to image asset paths. The catalog layer loads and validates the definitions when the back-end starts, so invalid content is rejected before the application starts.

The implementation also considers security concerns. Image files referenced by the IAT definitions can be located anywhere, but at startup each image is assigned a unique hash based on the location path. This means that the file-system path is not directly exposed on the internet. Additionally, only known hashes can be retrieved.

When a participant selects an IAT, the session API gets the matching definition and generates a deterministic run plan, either from a random seed or a selected seed for evaluation runs. The run plan contains the block sequence, response labels, stimuli, and the correct response sides. The session metadata is stored before the plan is sent to the front-end. This makes it possible to later perform analysis based on what the participant was actually assigned, even if the implementation is later updated.

7.2 Front-End Flow

The front-end is implemented as a TypeScript single-page application with manual rendering. It does not use any front-end frameworks for rendering. This keeps the timing-sensitive parts easier to inspect.

In line with previous findings (Rodd 2024), the participant is given clear instructions on how to interact with the application. The layout of the website aims to be concise and responsive, offering the user a clear indication of how to navigate it. The actual presentation of the site remains quite minimal to prevent distracting elements from affecting the participant’s ability to participate in the tests. During a test, the user is always informed about which actions they should take during each trial, when the test begins, and when it has ended.

When the application loads, it creates the runtime state, binds input listeners, requests the available IATs from the back-end, and renders the catalog view. When a participant selects an IAT, the front-end requests a new session and receives the generated plan. At this point, the participant can review the stimuli while they are being preloaded. When loading is complete, they are allowed to proceed.

During the blocks, the front-end presents the trials continuously. The participant can respond using a keyboard on desktop devices and using touch input on touch devices. The front-

end only allows touch control if a CSS media query determines that touch is the primary interaction method. Conversely, only keyboard input is allowed if keyboard input is the primary interaction method. This rules out variance caused by participants using a mouse or touchpad on desktops. The interface also adapts by showing the interaction needed to respond to each side.

7.3 Timing and Block Uploads

Response timing is handled in the front-end, while network communication is kept outside the trials. Once a trial is rendered, the client waits for two animation frames before recording the trial-start timestamp. This `requestAnimationFrame` loop approach places the timestamp closer to when the browser paints, rather than just the moment the internal state was changed. Timestamps are recorded using `performance.now()`, which is a high-resolution monotonic timer (MDN Web Docs Contributors 2026c).

While trials are running, the responses are recorded as trial events. When the first correct response is made, the trial is locked and the timestamp is recorded so that later events cannot overwrite the measured latency. Incorrect responses are stored as part of the event history.

The response time is calculated as the difference between the trial start timestamp and the timestamp of the response. After a trial ends, the next stimulus is delayed by 120 ms. This delay is used to make transitions slightly less abrupt.

Completed trial data is kept locally until the current block has finished. The completed block is then uploaded using the session API in one request. This keeps network communication between blocks rather than inside trials.

7.4 Persistence and Scoring

The database stores both the generated plans and the recorded trial events. The database is implemented with SQLite using SQLAlchemy, and the session repositories define functions for interacting with the database.

When a completed block is uploaded, the payload is validated against the database to ensure the submitted data is valid. The validated data is then persisted. After all blocks have been completed, the client requests the score. The back-end computes the score on demand by loading the stored plan and the trial events, and then calculating the participant's D-score using the D2 convention described in the background chapter.

7.5 Evaluation Tooling

The evaluation tooling is implemented as a separate application that starts browser sessions through Selenium Grid and automates the participant-facing application through ordinary inputs. Each run is configured with an IAT, plan seed, response delay, CPU throttling settings, and network degradation settings.

The evaluation application uses Selenium Grid to start and control browsers on the available workers. Selenium is used to execute JavaScript in the browser. At the start of each benchmark condition, the evaluation application injects a harness script into the page. This harness sets up the browser-side code needed to run benchmark sessions. Individual sessions are then started by calling the run function exposed by the harness script, and the function returns the result when the benchmark is completed.

This design avoids sending separate Selenium commands for every block transition and trial response. Those commands would have to travel over the network from the Selenium client to the browser, which would add variance. By keeping the session control logic inside the browser, the benchmark reduces external control overhead and keeps the measured time closer to the application runtime behavior.

The browser-side harness interacts only with the application through ordinary inputs. It observes the state through the interface the front-end publishes. Whenever the state changes, the front-end publishes an event. Using events avoids polling, which simplifies the logic and minimizes operations.

Running each session individually through the browser-side harness avoids issues that occurred when executing long-running benchmarks on Android. Instead of using one Selenium script call for all runs, the system executes multiple script calls.

7.6 Build and Dependency Management

Bazel is used to manage builds, tests, and dependencies. This matters because the benchmark results depend on running the same front-end, back-end, and evaluation code across repeated runs. Building the application with Bazel makes the evaluation less dependent on manual setup steps and undeclared local files.

The current build setup has only been tested on Linux. macOS is expected to work, but this has not been verified. Running the build directly on Windows is not supported.

8 Evaluation

This chapter evaluates the implemented application using the data produced by the analysis pipeline described in the methodology. The evaluation focuses on whether the implementation keeps the timing-critical code stable with different stimulus types, delays, platforms, and environmental pressure.

8.1 Experimental Setup

The final dataset contains six benchmark conditions, 26 worker runs, 260 completed sessions, and 22,880 completed trials. All 22,880 client-recorded response latencies fell within the dispatch window recorded by the evaluator. This is important because it shows that the application timing and the evaluator timing agree for every recorded trial.

The benchmark covers three baseline IAT configurations, one longer response delay, one CPU-throttled condition, and one network-degraded condition. The baseline conditions and longer response delay were run on five workers across Android, Linux and Windows using Chrome and Firefox. CPU throttling and network degradation were limited to Android, Linux and Windows using Chrome because of the limitation mentioned in the methodology chapter.

Each worker completed ten sessions for each of its assigned conditions. The plan seed was fixed to 123 for all benchmark conditions, so sessions use the same generated trial structure.

8.2 Condition-Level Results

Table 3 summarizes the client-response latencies and the mean duration of a worker run. The latency values show what the participant-facing application persisted as the response time for each trial. The run duration shows the mean time for one worker to complete its ten-session benchmark run for that condition. This is useful to measure because conditions with a longer delay or network degradation should also affect the session duration. This is therefore part of validating that the conditions ran correctly. This table, by itself, does not show application-induced timing variance, because, as mentioned in the methodology, the response delay is a best-effort delay.

The latency table is mainly a logic check that the benchmark ran as expected. The response latencies for all conditions were close to the configured response delay. This is important so that it does not become an uncontrolled variable in comparisons.

The run durations are also useful as a validation measure. The three baseline conditions

Condition	Trials	Mean	SD	Min	Max	Mean run duration
Text/text baseline	4,400	410.30	1.41	409.0	460.0	9:07 min
Image/text baseline	4,400	410.26	1.32	409.0	446.0	9:08 min
Image/image baseline	4,400	410.26	1.36	409.0	450.0	9:09 min
1000 ms delay	4,400	1005.67	11.24	999.0	1088.0	17:54 min
CPU throttling	2,640	411.56	6.95	409.1	742.8	9:28 min
Network degradation	2,640	410.18	1.13	409.1	436.4	16:39 min

Table 3: Condition-level client latency summary. Mean, standard deviation, minimum, and maximum refer to the client-recorded latency in milliseconds. Run duration is the mean duration of one worker’s ten-session benchmark run for the condition.

took practically the same time to complete, while the 1000 ms delay and network-degraded conditions took longer. This supports that the benchmark conditions were applied.

Table 4 shows the relationship between the evaluator’s recorded input window and the latency persisted by the application. The *DB minus before* column is the difference between the participant-facing client’s recorded latency and the evaluator timestamp recorded immediately before sending the input. The *after minus DB* column is the difference between the evaluator timestamp recorded immediately after sending the input and the client’s persisted latency.

Condition	DB minus before	After minus DB	Dispatch
Text/text baseline	0.09 (0.17)	0.48 (0.67)	0.57 (0.78)
Image/text baseline	0.09 (0.18)	0.64 (0.90)	0.73 (1.00)
Image/image baseline	0.09 (0.18)	0.73 (1.00)	0.82 (1.11)
1000 ms delay	0.10 (0.19)	0.66 (0.88)	0.75 (0.97)
CPU throttling	0.45 (0.81)	2.96 (3.65)	3.40 (4.15)
Network degradation	0.12 (0.15)	1.21 (1.63)	1.33 (1.75)

Table 4: Client-to-evaluator timing comparison. Timing values are shown as mean (SD) in milliseconds.

The baseline and 1000 ms delay conditions show that the client latency remains quite stable. This is more important than the timing of the client latency table alone because it compares what the application stored with when the evaluator actually sent the input.

CPU throttling is the condition where the timing variance becomes noticeably worse. The dispatch window, *DB minus before*, and *after minus DB* all become roughly four to five times as large. The wider dispatch window indicates that the browser under load becomes slower and the increase in dispatch standard deviation indicates that the browser becomes less consistent.

Network degradation had minimal effect on *DB minus before*, but did have some effect on *after minus DB*. This suggests that the persisted client latency remains close to the evaluator’s before-input timestamp, while the input dispatch window becomes slightly wider under degraded network conditions. The benchmark does not isolate the exact cause of the wider *after minus DB* window, but the effect remains small compared with CPU throttling. The result still suggests that network pressure mainly affects work outside the timing-critical trial loop.

8.3 Latency and Timing Variance Benchmarks

8.3.1 Baseline Stimulus Conditions

The three baseline conditions compare whether text and image stimuli change timing variance during the trial loop. The result is that they behave almost the same. Using image stimuli did not noticeably increase latency, variance, or session duration.

This result is relevant because image stimuli would be expected to create extra variance if images were loaded or decoded during the trial loop. The result instead supports the conclusion that stimulus preloading kept image handling outside of the timing-sensitive parts.

The per-worker results still show some platform and browser differences. However, this does not change the overall result, as it shows that the application limits avoidable variance in the trial loop and that browser and platform remain part of the variance.

8.3.2 CPU Throttling

The CPU-throttled condition used an image/text IAT with a 410 ms response delay and a browser CPU throttling rate of 10.

For CPU throttling, the *DB minus before* value remains small, which means the application still records the response close to the evaluator’s before-input timestamp. Its variance still increased, but the larger effect was seen in the change of *after minus DB*. The client therefore still records the response close to the event, but CPU pressure introduces more variance around input dispatch and event handling.

CPU throttling also did not affect all workers equally. Table 5 shows that the effect on Android Chrome is especially noticeable, where the *DB minus before* variance becomes much larger.

Table 5 shows CPU-throttled client-to-evaluator comparisons for each worker. The difference between workers is important because the condition-level summary hides where the variation

comes from. Android Chrome had the widest dispatch window, while Windows Chrome did not perform much worse under stress than it did without stress.

Worker	DB minus before	After minus DB	Dispatch
Android Chrome	1.21 (1.01)	7.62 (2.48)	8.84 (2.43)
Linux Chrome	0.10 (0.27)	1.11 (0.85)	1.21 (0.86)
Windows Chrome	0.03 (0.09)	0.14 (0.28)	0.17 (0.35)

Table 5: Per-worker evaluator-to-database timing comparison under CPU throttling. Values are shown as mean (SD) in milliseconds.

8.3.3 Network Degradation

The network-degraded condition used an image/text IAT with a 410 ms response delay, 500 kbps download throughput, 500 kbps upload throughput, 220 ms latency, and disabled caching.

The network degradation condition mainly affected run duration, which became much longer, showing that the network constraints were applied. It had minimal effect on *DB minus before*, but did have some effect on *after minus DB*. This means that the recorded client latency still occurs near the start of the evaluator’s input, while the input dispatch takes slightly longer to complete.

8.3.4 Longer Delay Stability Evaluation

The longer delay increases the run duration, which is expected because the added wait time accumulates across trials. The client-recorded latency spread is larger than in the baseline, while the client-to-evaluator variance remains small, indicating that the system still performs correctly.

Table 6 shows the per-worker comparison of the 1000 ms delay condition. The Firefox workers had slightly higher variance, although they remained reasonable. Android Chrome showed noticeably higher *after minus DB*, which means the evaluator took longer to complete after the input action.

This supports the conclusion that the timing behavior remains stable during the longer 1000 ms-delay benchmark.

Worker	DB minus before	After minus DB	Dispatch
Android Chrome	0.29 (0.08)	2.21 (0.59)	2.50 (0.61)
Linux Chrome	0.05 (0.06)	0.25 (0.09)	0.29 (0.10)
Linux Firefox	0.07 (0.26)	0.42 (0.52)	0.49 (0.53)
Windows Chrome	0.03 (0.05)	0.15 (0.08)	0.18 (0.08)
Windows Firefox	0.05 (0.22)	0.26 (0.45)	0.31 (0.47)

Table 6: Per-worker evaluator-to-database timing comparison with 1000 ms delay. Values are shown as mean (SD) in milliseconds.

9 Discussion

9.1 Interpretation of Results

The benchmark runs support the thesis claim that a deterministic client-side architecture can reduce avoidable measurement variance in a web-based IAT application.

The comparison between the text/text, image/text, and image/image baseline conditions is the clearest result. If image resources were being fetched or decoded during the timing-critical trials, there should have been a larger increase in latency and variance. Instead, the differences between the baseline client-to-evaluator values were minimal. This suggests that image preloading moved the main variance out of the trial loop.

The network-degraded condition also supports this interpretation. The run duration became much longer, showing that the network degradation had an effect. At the same time, the trial latency variance stayed close to the baseline conditions. This suggests that the network mainly affects loading, uploads, and other work outside the timing-critical trials. This is important because the architecture was meant to keep network communication outside the timing-sensitive code.

The CPU-throttled condition shows that the variance becomes noticeably larger, especially on Android Chrome. This does not, however, contradict the architecture, since the CPU pressure affects the browser event loop, rendering, and input handling. This is part of the client environment that cannot be fully controlled by the application. It is important to note that even though the variance was noticeably worse, the variance compared to just before the evaluator’s input still only had a standard deviation of 1.01 ms on the device that performed the worst. The result therefore shows that the architecture reduces avoidable variance from the application, but does not remove device-related environmental variance.

The 1000 ms delay condition is useful because it shows the application working even under longer sessions. Increasing the response delay by 590 ms did increase the session duration

substantially, but did not increase latency variance. This shows that the client is stable even with longer sessions during which the browser remains active for longer periods.

Overall, the results lead to the following answer to the research question. A deterministic client-side architecture reduces avoidable measurement variance in a web-based IAT application by removing asset loading, server-side decision-making, and network communication from the trial loop. The architecture does not eliminate all variance, but shifts the variance toward browser scheduling, hardware, and input devices.

9.2 Limitations and Future Work

The main methodological limitation is that the evaluation used automated responses rather than human participants. This was useful for measuring the application-induced variance, because it kept the responses controlled. However, it also means that the evaluation does not determine whether the interface produces valid psychological data. This would require a participant study.

The timing measurements are also limited by how the evaluation was done. The evaluator records a window around the input event and compares this window against the latency recorded by the client. This shows only the timing variance between the evaluator and the client, but is not the same as measuring the absolute latency from input hardware to display hardware. The implementation uses `performance.now()` and starts the trial post-paint, but exactly when the stimuli physically appear is outside of what this benchmark measures. Future work could therefore include external hardware-based display and input measurements.

The platform coverage does not include all possible platforms. The benchmarks covered workers on Windows, Linux, and Android using Chrome and Firefox. They did not cover Safari, iOS, and many different types of hardware. Since CPU throttling showed larger differences between workers, broader device coverage could show different results.

The environmental pressure was simulated. Simulated CPU throttling and network degradation are useful because they make controlled comparisons possible, but they do not represent real conditions. A participant's device could be affected by different factors such as browser extensions, thermal throttling, or other device-specific configurations that the benchmark does not include.

The stimulus material was generated because the focus is on the software architecture rather than the psychological aspect. This makes the system easier to distribute and reproduce,

but the stimuli should not be treated as psychologically validated material. A deployment intended for real behavioral research would therefore need validated stimuli.

The automated evaluation used fixed plan seeds and always selected the correct responses. This was useful for isolating the timing behavior, but did not execute every path a real user might encounter. Incorrect responses and different plans could therefore be studied further.

10 Conclusion

This thesis set out to answer the following question: how can a deterministic client-side architecture minimize measurement variance in web-based Implicit Association Tests across heterogeneous hardware and varying network conditions? The implemented system addresses this by moving the timing-critical trial loop entirely into the browser after the back-end has created a plan and the client has preloaded the required stimuli. During trials, response times are measured using a high-resolution monotonic timer, while network communication is avoided inside the trial loop.

The benchmark conditions indicate that the implemented system reduces avoidable variance. Across the three 410 ms baseline conditions, text/text, image/text, and image/image, the differences between client-recorded latencies and evaluator input timestamps had standard deviations between 0.17 ms and 0.18 ms. This indicates that the use of image stimuli did not noticeably increase timing variance in the tested implementation, which supports the architectural decision to preload stimuli.

The stressed conditions show what the architecture cannot fully control. CPU throttling increased timing variance and dispatch overhead, showing that browser scheduling and device performance remain important sources of measurement uncertainty. Network degradation affected the overall session duration, but had only a limited effect on the timing comparison between the evaluator and the client during trials.

The project therefore shows that a deterministic client-side IAT architecture can limit avoidable measurement variance introduced by the application itself. The remaining measurement variance is not eliminated but is instead mainly associated with factors outside the direct control of the application, such as browser scheduling, device characteristics, display behavior, and input hardware.

Bibliography

- Anwyl-Irvine, Alexander et al. (2021). “Realistic precision and accuracy of online experiment platforms, web browsers, and devices”. In: *Behavior Research Methods* 53.4, pp. 1407–1425. DOI: 10.3758/s13428-020-01501-5.
- Bridges, David et al. (2020). “The timing mega-study: comparing a range of experiment generators, both lab-based and online”. In: *PeerJ* 8, e9414. DOI: 10.7717/peerj.9414.
- Cuddy, Amy JC, Susan T Fiske, and Peter Glick (2007). “The BIAS map: behaviors from intergroup affect and stereotypes.” In: *Journal of personality and social psychology* 92.4, p. 631.
- Dasgupta, Nilanjana and Anthony G Greenwald (2001). “On the malleability of automatic attitudes: combating automatic prejudice with images of admired and disliked individuals.” In: *Journal of personality and social psychology* 81.5, p. 800.
- De Leeuw, Joshua R (2015). “jsPsych: A JavaScript library for creating behavioral experiments in a Web browser”. In: *Behavior research methods* 47.1, pp. 1–12.
- Ecma International (2025). *ECMAScript® 2025 Language Specification*. Ecma International. URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/> (visited on 06/01/2026).
- Forscher, Patrick S et al. (2019). “A meta-analysis of procedures to change implicit measures.” In: *Journal of personality and social psychology* 117.3, p. 522.
- Gao, Zhen et al. (2020). “Implementation of stimuli with millisecond timing accuracy in online experiments”. In: *PLoS ONE* 15.7, e0235249. DOI: 10.1371/journal.pone.0235249.
- Goldstone, Robert L, Alan Kersten, and Paulo F Carvalho (2003). “Concepts and categorization”. In: *Comprehensive handbook of psychology* 4, pp. 599–621.
- Greenwald, Anthony G and Mahzarin R Banaji (1995). “Implicit social cognition: attitudes, self-esteem, and stereotypes.” In: *Psychological review* 102.1, p. 4.
- Greenwald, Anthony G., Debbie E. McGhee, and Jordan L. K. Schwartz (1998). “Measuring individual differences in implicit cognition: The implicit association test”. In: *Journal of Personality and Social Psychology* 74.6, pp. 1464–1480. DOI: 10.1037/0022-3514.74.6.1464.
- Greenwald, Anthony G., Brian A. Nosek, and Mahzarin R. Banaji (2003). “Understanding and using the Implicit Association Test: I. An improved scoring algorithm”. In: *Journal of Personality and Social Psychology* 85.2, pp. 197–216. DOI: 10.1037/0022-3514.85.2.197.

- Haselton, Martie G, Daniel Nettle, and Paul W Andrews (2015). “The evolution of cognitive bias”. In: *The handbook of evolutionary psychology*, pp. 724–746.
- Leeuw, Joshua R. de and Benjamin A. Motz (2016). “Psychophysics in a Web browser? Comparing response times collected with JavaScript and Psychophysics Toolbox in a visual search task”. In: *Behavior Research Methods* 48.1, pp. 1–12. DOI: 10.3758/s13428-015-0567-2.
- Lukács, Gáspár and Andreas Gartus (May 2022). “Precise display time measurement in JavaScript for web-based experiments”. In: *Behavior Research Methods* 55. DOI: 10.3758/s13428-022-01835-2.
- MDN Web Docs Contributors (2026a). *In depth: Microtasks and the JavaScript runtime environment*. Mozilla Developer Network. URL: https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API/Microtask_guide/In_depth (visited on 06/01/2026).
- (2026b). *JavaScript execution model*. Mozilla Developer Network. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model (visited on 06/01/2026).
- (2026c). *Performance: now() method*. Mozilla Developer Network. URL: <https://developer.mozilla.org/en-US/docs/Web/API/Performance/now> (visited on 06/01/2026).
- Reimers, Stian and Neil Stewart (June 2014). “Presentation and response timing accuracy in Adobe Flash and HTML5/JavaScript Web experiments”. In: *Behavior research methods* 47. DOI: 10.3758/s13428-014-0471-1.
- Rodd, Jennifer M. (2024). “Moving experimental psychology online: How to obtain high quality data when we can’t see our participants”. In: *Journal of Memory and Language* 134, p. 104472. ISSN: 0749-596X. DOI: <https://doi.org/10.1016/j.jml.2023.104472>. URL: <https://www.sciencedirect.com/science/article/pii/S0749596X23000712>.
- Rotem-Gal-Oz, Arnon (2006). “Fallacies of distributed computing explained”. In: URL <http://www.rgoarchitects.com/Files/fallacies.pdf> 20, p. 26.
- Tajfel, Henri (1970). “Experiments in intergroup discrimination”. In: *Scientific american* 223.5, pp. 96–103.

11 Appendices

11.1 Source code and evaluation data

The source code of the implemented system and evaluation tooling is available at <https://github.com/TrapsterDK/IAT>. The repository also contains the benchmark con-

figurations and the evaluation data used in this report. The version used for the submitted thesis is commit 15547845b62ea754cb4c845e99edbadcf4fcbda7.

11.2 Usage of GenAI

Throughout the project, LLMs were used to support the workflow. The following LLMs were used: GPT-5.4, GPT-5.5, Gemini 3, and GitHub Copilot. The tools were used for the following:

- Identifying possible sources, which were manually checked before being used.
- Researching possible solutions to code problems.
- Idea generation for topics such as development patterns and frameworks.
- Generating diagrams in LaTeX from text.
- Grammar checking.
- Code assistance with GitHub Copilot.

Another use of generative AI tools was the generation of visual stimuli, and the rationale behind this decision has already been discussed in the methodology chapter.